

Ulysses Phase 1 Proposal v2

Kenneth Lew

January 2026

1 Introduction

The control algorithm can be separated into 4 main modules:

1. **Body Torque Control** – converts attitude error into torque command ($\vec{\tau}_{cmd}$).
2. **Control Allocation** – decomposes the torque command into torque command to the gimbal ($\vec{\tau}_{gim}$) and torque command to the propellers ($\vec{\tau}_{thrust}$) to ensure full .
3. **Gimbal Control** – converts torque command and thrust magnitude command to gimbal angles for the servos.
4. **Thrust Control** – generates thrust magnitude (T) command based on the error in z . Then it combines with $\vec{\tau}_{thrust}$ to map (T_{cmd}, τ_{thrust}) to (PWM_1, PWM_2) for the propeller.

IMPORTANT!

1. Note that we will be utilizing a z-axis up convention, however, it can be adapted to NED if needed.
2. All quaternions must be unit for the calculations to work.

2 Nomenclature

- $\vec{x}$ - 3×1 position in the world frame.
- $\vec{v}$ - 3×1 velocity in the world frame.
- $\mathbf{q}$ - unit world to body quaternion representing the frame of the rocket (attitude). A unit quaternion can be written in the form $\mathbf{q} = q_0 + q_1\hat{i} + q_2\hat{j} + q_3\hat{k}$.

- $\Delta \mathbf{q}$ - quaternion error (unit).
- $\mathbf{q}_{ref}$ - desired attitude quaternion (unit).
- $\mathbf{I}$ - 3×3 centred moment of inertia matrix in the body frame.
- $\vec{\phi}$ - 3×1 attitude error.
- $\vec{\omega}$ - 3×1 body angular velocity in the body frame.
- $\vec{\tau}$ - 3×1 torque on the body.
- $\vec{T}$ - 3×1 thrust vector.
- $[\theta_x \ \theta_y]^\top$ - gimbal angles around x axis and y axis, respectively.

3 Torque Module

The torque module computes the required rotation ($\Delta \mathbf{q}$) to get the rocket from the current attitude ($\mathbf{q}$) to the desired attitude ($\mathbf{q}_{ref}$). This given by the following formula¹:

$$\Delta \mathbf{q} = \mathbf{q}_{ref} \otimes \mathbf{q}^* \quad (3.1)$$

where $\Delta \mathbf{q}$ is the quaternion error. In case that the error quaternion is demanding a rotation more than π radians, the closest rotation is the inverted direction and this is found by examining the real part of the quaternion, q_0 . If $q_0 < 0$ then the desired orientation is more than π radians away and the closest rotation is the negative of $\Delta \mathbf{q}$.

Note that the error quaternion's vector part $\vec{u}$ and the torque on the rocket both carry a direction that is meaningful in the right-hand-rule (RHR) sense:

- $\hat{u}$ points along the RHR rotation axis of the attitude error.
- $\vec{\tau}$ points along the RHR axis about which angular momentum is being changed, since $\vec{\tau} = \dot{\vec{L}}$.

Thus, to directly reduce the rotation associated with the error axis $\hat{u}$, the corrective torque must have a component along $\hat{u}$. Any torque perpendicular to $\hat{u}$ introduces precession rather than directly undoing rotation about $\hat{u}$. Let ϕ be the attitude error angle and $\hat{u}$ its axis, and define the axis-angle error as follows:

$$\vec{\phi} := \phi \hat{u}. \quad (3.2)$$

A unit quaternion representing a rotation by angle ϕ about axis $\hat{u}$ can be written as:

$$\mathbf{q}(\phi, \hat{u}) = \left[\cos \frac{\phi}{2}, \hat{u} \sin \frac{\phi}{2} \right] \quad (3.3)$$

¹ $\otimes$ and $(\cdot)^*$ denote the quaternion product and the quaternion conjugate, respectively.

For small attitude errors, $\sin(\phi/2) \approx \phi/2$ and $\cos(\phi/2) \approx 1$, so the error quaternion satisfies:

$$\begin{aligned}\Delta \mathbf{q} &\approx \left[1, \frac{\phi}{2} \hat{u} \right] \\ &= \left[1, \frac{1}{2} \vec{\phi} \right]\end{aligned}\tag{3.4}$$

Combining (3.2) and (3.4), we get that the quaternion vector part is approximately proportional to the axis-angle error:

$$\vec{\phi} \approx 2\vec{u}.\tag{3.5}$$

By Euler's rotation equations, the rigid-body rotational dynamics are given by the following equation:

$$\vec{\tau}_{ext} = \mathbf{I}\dot{\vec{\omega}} + \vec{\omega} \times (\mathbf{I}\vec{\omega}).\tag{3.6}$$

We choose the following PD law for the desired closed-loop rotational dynamics:

$$\mathbf{I}\dot{\vec{\omega}} = -\mathbf{K}_p\vec{\phi} - \mathbf{K}_d\vec{\omega}\tag{3.7}$$

where $\mathbf{K}_p, \mathbf{K}_d$ are the proportional and derivative gains respectively. Depending on our needs, the gains can either be scalar, a diagonal matrix, or a full 3×3 matrix with cross-terms. Finally, substituting (3.7) into (3.6) gives the required control torque, $\vec{\tau}_{cmd}$:

$$\boxed{\vec{\tau}_{cmd} = -\mathbf{K}_p\vec{\phi} - \mathbf{K}_d\vec{\omega} + \vec{\omega} \times (\mathbf{I}\vec{\omega})}\tag{3.8}$$

The first term applies a restoring torque along the error axis, the second damps the angular rate, and the $\vec{\omega} \times (\mathbf{I}\vec{\omega})$ term provides feedforward compensation that cancels the gyroscopic coupling.

4 Torque Control Allocation

Let $\vec{\tau}_{cmd}$ be the total body torque demanded by the attitude controller. In our vehicle, the available actuators do not span all torque directions equally. The gimbal can only generate lateral torque (ie. in the x and y directions). This means that we cannot generate torque axially (in z direction), which could be a problem because the weight of the gimbal is not negligible and could potentially generate axial torque on the body.

The tentative proposal to mitigate potential axial torque is to utilize the differential propeller pair. By differentially commanding the two counter-rotating props, the couple moment in the thrust direction can theoretically be controlled to get the desired axial torque.

For this reason, we split the command into a part assigned to the gimbal and a part assigned to the thruster:

$$\vec{\tau}_{cmd} = \vec{\tau}_{gim} + \vec{\tau}_{thrust}\tag{4.1}$$

The torque in the z direction is given by:

$$\tau_z = \vec{\tau}_{cmd} \cdot \hat{k} \quad (4.2)$$

We can then find the amount of torque the differential propeller pair should generate by using the following formula:

$$\boxed{\tau_{thrust} := \frac{\tau_z}{t_z}} \quad (4.3)$$

where t_z is the z component of the unit thrust direction $\hat{t} = [t_x \quad t_y \quad t_z]^\top$.

Finally, we subtract $\vec{\tau}_{thrust}$ from $\vec{\tau}_{cmd}$ to get $\vec{\tau}_{gim}$:

$$\boxed{\vec{\tau}_{gim} = \vec{\tau}_{cmd} - \tau_{thrust}\hat{t}} \quad (4.4)$$

5 Gimbal Control

With the gimbal pivot located a distance L from the vehicle's center of mass along the body z-axis, the thrust vector $\vec{T} = [T_x, T_y, T_z]^\top$ exerts a torque given by:

$$\vec{\tau}_{gim} = \vec{r}_{gim} \times \vec{T} \quad (5.1)$$

where $\vec{r}_{gim} := [0 \quad 0 \quad L]^\top$. Expanding the cross product yields:

$$\vec{\tau}_{gim} = [-LT_y \quad LT_x \quad 0]^\top \quad (5.2)$$

so matching a desired torque (τ_x, τ_y) implies the lateral thrust components:

$$T_x = \frac{\tau_y}{L}, \quad T_y = -\frac{\tau_x}{L} \quad (5.3)$$

If the thruster magnitude is fixed to $T = \|\vec{T}\|$, then the axial component is determined by the norm constraint,

$$T_z = \sqrt{T^2 - T_x^2 - T_y^2}, \quad (5.4)$$

which is feasible only when $T_x^2 + T_y^2 \leq T^2$. This construction enforces the commanded body torques while respecting the actuator thrust limit.

Define the unit thrust direction in the body frame:

$$\hat{t} := \frac{\vec{T}}{T} = [t_x \quad t_y \quad t_z]^\top$$

Using (5.3) and (5.4), we get that:

$$\hat{t} = \left[\frac{\tau_y}{LT} \quad -\frac{\tau_x}{LT} \quad \sqrt{1 - t_x^2 - t_y^2} \right]^\top \quad (5.5)$$

Next, we extract the gimbal angles θ_x, θ_y from the thrust direction $\hat{t}$. We assume the convention of first applying the rotation θ_y about body y, then the rotation θ_x about body x to the downward direction $-\hat{k}$. By this convention:

$$\hat{t} = R_x(\theta_x)R_y(\theta_y)(-\hat{k}) = \begin{bmatrix} -\sin \theta_y \\ \sin \theta_x \cos \theta_y \\ -\cos \theta_x \cos \theta_y \end{bmatrix} \quad (5.6)$$

Extracting the angles from (5.6), we get that:

$$\theta_x = \text{atan2}(t_y, -t_z), \quad \theta_y = -\arcsin(t_x) \quad (5.7)$$

(Optional) If we use the small approximation, we set $\sin \theta \approx \theta$ and $\cos \theta \approx 1 - \frac{\theta^2}{2}$. Then (5.6) becomes:

$$\hat{t} \approx \begin{bmatrix} -\theta_y \\ \theta_x \\ -1 + \frac{1}{2}(\theta_x^2 + \theta_y^2) \end{bmatrix} \quad (5.8)$$

Combining this with (5.5), we thus get:

$$\theta_x \approx -\frac{\tau_x}{LT}, \quad \theta_y \approx -\frac{\tau_y}{LT} \quad (5.9)$$

Finally, θ_x and θ_y command is saturated to respect actuator limits:

$$\theta_{cmd} \leftarrow \text{clip}(\theta_{cmd}, \theta_{\min}, \theta_{\max}). \quad (5.10)$$

6 Thrust Control

For phase 1, we will be using a simple 1D PID controller on the z position of the rocket to command the thrust magnitude of the rocket. Compute position error as:

$$e_z = z_{ref} - z \quad (6.1)$$

with error derivative $\dot{e}_z = \dot{z}_{ref} - \dot{z}$. The controller outputs a commanded vertical acceleration:

$$a_{z,cmd} = K_p e_z + K_d \dot{e}_z + K_i \int e_z dt \quad (6.2)$$

Using the vertical translational dynamics $m\ddot{z} = T - mg$ (with $+z$ upward), the required thrust command is:

$$T_{cmd} = m(g + a_{z,cmd}) \quad (6.3)$$

where m is the mass of the rocket and $g = 9.8067 \text{ ms}^{-2}$ is the gravitational acceleration. Finally, the thrust command is saturated to respect actuator limits:

$$T_{cmd} \leftarrow \text{clip}(T_{cmd}, T_{\min}, T_{\max}). \quad (6.4)$$

7 Appendix

